

Set 2 - Endsem Lab Exam - Set A

Instructions:

1. Attempt **any one** of the questions given below.
2. Use only the following editors: nano, vim, gedit, or nedit.
3. You can write your code in C/C++.
4. File naming in this format - **SetNo_QNo_RollNo.c/cpp**, e.g. 2_1_2025123.c or 2_1_2025123.cpp
5. **Google Form Link For Submission:** <https://forms.gle/sV8VeqWEg5tfpAp9>

Question 1: Implement Priority Queue using Heap

Write a C program to implement a **Priority Queue** using a **Max Heap** with an array-based implementation. The heap must satisfy the max-heap property, where every parent node is greater than or equal to its children. All operations must be performed using the array representation of a binary heap. The priority queue should support `insert(x)` for inserting an element into the priority queue, `heapMaximum()` for returning the highest priority element without removing it, and `heapExtractMax()` for removing and returning the element with the highest priority. Use the helper function `heapIncreaseKey(i, key)` to increase the value of an element at index `i` to a new key and restore the max-heap property by moving the element upward during insertion.

Base Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #define NEG_INF -1000000000
4 struct Heap {
5     int* array;
6     int capacity;
7     int size;
8 };
9 struct Heap* createHeap(int capacity) {
10     struct Heap* heap = (struct Heap*)malloc(sizeof(struct Heap));
11     heap->array = (int*)malloc(capacity * sizeof(int));
12     heap->capacity = capacity;
13     heap->size = 0;
14     return heap;
15 }
16 void swap(int* a, int* b) {
17     int temp = *a;
18     *a = *b;
19     *b = temp;
20 }
21 int heapMaximum(struct Heap* heap) {
22     /* Write your code here */
23     return -1;
24 }
25 int heapExtractMax(struct Heap* heap) {
26     /* Write your code here */
27     return -1;
28 }
29 void heapIncreaseKey(struct Heap* heap, int index, int key) { /* Write your code here */}
30
31 void insert(struct Heap* heap, int value) { /* Write your code here */}
32
33 void maxHeapify(struct Heap* heap, int i) {
34     int largest = i;
35     int left = 2 * i + 1;
36     int right = 2 * i + 2;
37
38     if (left < heap->size && heap->array[left] > heap->array[largest])
39         largest = left;
40
41     if (right < heap->size && heap->array[right] > heap->array[largest])
42         largest = right;
43
44     if (largest != i) {
45         swap(&heap->array[i], &heap->array[largest]);
46         maxHeapify(heap, largest);
47     }
48 }
49 void printHeap(struct Heap* heap) {
50     for (int i = 0; i < heap->size; i++)
51         printf("%d ", heap->array[i]);
52     printf("\n");
53 }
54 int main() {
55     struct Heap* heap = createHeap(100);
56     insert(heap, 40);
57     insert(heap, 20);
58     insert(heap, 50);
59     insert(heap, 10);
60     insert(heap, 30);
61
62     printf("Heap: ");
63     printHeap(heap);
64
65     printf("Max: %d\n", heapMaximum(heap));
66
67     printf("Extract: %d\n", heapExtractMax(heap));
68     printHeap(heap);
69
70     printf("Extract: %d\n", heapExtractMax(heap));
```

```

71 printHeap(heap);
72 return 0;}

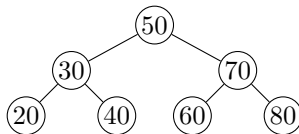
```

Question 2: Delete Node from Binary Search Tree (BST)

Write a C program to perform **deletion of a node in a Binary Search Tree (BST)**. The deletion must be implemented using a helper function `transplant(T, u, v)`, which replaces the subtree rooted at node `u` with the subtree rooted at node `v`. Your program must correctly handle all the cases, which include **leaf node**, **node with one child**, and **node with two children**.

Use the following operations to test your program:

1. Insert: 50, 30, 70, 20, 40, 60, 80
2. Delete: 20 (leaf node)
3. Delete: 30 (node with one child)
4. Delete: 50 (node with two children)



Base Code

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct Node{
5     int key;
6     struct Node *left,*right,*p;
7 } Node;
8 typedef struct Tree{
9     Node *root;
10 } Tree;
11
12 Node* newNode(int key){
13     Node* n=(Node*)malloc(sizeof(Node));
14     n->key=key; n->left=NULL; n->right=NULL; n->p=NULL;
15     return n;}
16
17 void insertNode(Tree* T,int key){
18     Node* z=newNode(key);
19     Node* x=T->root; Node* y=NULL;
20     while(x!=NULL){
21         y=x;
22         if(z->key<x->key) x=x->left;
23         else x=x->right;
24     }
25     z->p=y;
26     if(y==NULL) T->root=z;
27     else if(z->key<y->key) y->left=z;
28     else y->right=z;}
29
30 Node* treeSearch(Node* x,int k){
31     if(x==NULL || k==x->key) return x;
32     if(k<x->key) return treeSearch(x->left,k);
33     else return treeSearch(x->right,k);}
34
35 Node* treeMin(Node* x){
36     while(x->left!=NULL) x=x->left;
37     return x;}
38
39 void transplant(Tree* T,Node* u,Node* v){/* Write your code here */}
40
41 void treeDelete(Tree* T,Node* z){/* Write your code here */}
42
43 void inorder(Node* x){
44     if(x==NULL) return;
45     inorder(x->left);
46     printf("%d -> ",x->key);
47     inorder(x->right);}
48
49 int main(){
50     int i;
51     int arr[]={50,30,70,20,40,60,80};
52     int deleteKeys[]={20,30,50};
53
54     Tree T; T.root=NULL;
55
56     for(i=0;i<7;i++) insertNode(&T,arr[i]);
57
58     printf("Initial inorder traversal: ");
59     inorder(T.root); printf("NULL\n");
60
61     for(i=0;i<3;i++){
62         Node* z=treeSearch(T.root,deleteKeys[i]);
63         if(z==NULL){
64             printf("\nKey %d not found in the tree.\n",deleteKeys[i]);
65         } else {
66             printf("\nDeleting %d:\n",deleteKeys[i]);
67             treeDelete(&T,z);
68             printf("Inorder traversal after deletion: ");
69             inorder(T.root); printf("NULL\n");
70         }
71     }
72     return 0;}

```

Solution 1

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #define NEG_INF -1000000000
4
5 struct Heap {
6     int* array;
7     int capacity;
8     int size;
9 };
10
11 struct Heap* createHeap(int capacity) {
12     struct Heap* heap = (struct Heap*)malloc(sizeof(struct Heap));
13     heap->array = (int*)malloc(capacity * sizeof(int));
14     heap->capacity = capacity;
15     heap->size = 0;
16     return heap;
17 }
18
19 void swap(int* a, int* b) {
20     int temp = *a;
21     *a = *b;
22     *b = temp;
23 }
24
25 int heapMaximum(struct Heap* heap) {
26     if (heap->size <= 0) {
27         printf("Heap underflow\n");
28         return -1;
29     }
30     return heap->array[0];
31 }
32
33 void maxHeapify(struct Heap* heap, int i) {
34     int largest = i;
35     int left = 2 * i + 1;
36     int right = 2 * i + 2;
37
38     if (left < heap->size && heap->array[left] > heap->array[largest])
39         largest = left;
40
41     if (right < heap->size && heap->array[right] > heap->array[largest])
42         largest = right;
43
44     if (largest != i) {
45         swap(&heap->array[i], &heap->array[largest]);
46         maxHeapify(heap, largest);
47     }
48 }
49
50 int heapExtractMax(struct Heap* heap) {
51     int max;
52     if (heap->size <= 0) {
53         printf("Heap underflow\n");
54         return -1;
55     }
56
57     max = heap->array[0];
58     heap->array[0] = heap->array[heap->size - 1];
59     heap->size--;
60     maxHeapify(heap, 0);
61
62     return max;
63 }
64
65 void heapIncreaseKey(struct Heap* heap, int index, int key) {
66     if (key < heap->array[index])
67         return;
68
69     heap->array[index] = key;
70
71     while (index > 0 && heap->array[(index - 1) / 2] < heap->array[index]) {
72         int parent = (index - 1) / 2;
73         swap(&heap->array[index], &heap->array[parent]);
74         index = parent;
75     }
76 }
77
78 void insert(struct Heap* heap, int value) {
```

```

79     if (heap->size == heap->capacity) {
80         printf("Heap overflow\n");
81         return;
82     }
83
84     heap->size++;
85     heap->array[heap->size - 1] = NEG_INF;
86     heapIncreaseKey(heap, heap->size - 1, value);
87 }
88
89 void printHeap(struct Heap* heap) {
90     for (int i = 0; i < heap->size; i++)
91         printf("%d ", heap->array[i]);
92     printf("\n");
93 }
94
95 int main() {
96     struct Heap* heap = createHeap(100);
97
98     insert(heap, 40);
99     insert(heap, 20);
100    insert(heap, 50);
101    insert(heap, 10);
102    insert(heap, 30);
103
104    printf("Heap: ");
105    printHeap(heap);
106
107    printf("Max: %d\n", heapMaximum(heap));
108
109    printf("Extract: %d\n", heapExtractMax(heap));
110    printHeap(heap);
111
112    printf("Extract: %d\n", heapExtractMax(heap));
113    printHeap(heap);
114
115    return 0;
116 }

```

Rubric 1

Criteria	Marks
Correct program structure and function usage	1
Correct heap insertion and priority queue creation	1
Correct implementation of heapMaximum(), heapExtractMax(), and maxHeapInsert(), heapIncreaseKey()	4
Proper maintenance of max-heap property after each operation	3
Correct output	1
Total	10

Solution 2

```

1     #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct Node{
5     int key;
6     struct Node *left,*right,*p;
7 } Node;
8
9 typedef struct Tree{
10     Node *root;
11 } Tree;
12
13 Node* newNode(int key){
14     Node* n=(Node*)malloc(sizeof(Node));
15     n->key=key; n->left=NULL; n->right=NULL; n->p=NULL;
16     return n;
17 }
18
19 void insertNode(Tree* T,int key){
20     Node* z=newNode(key);
21     Node* x=T->root;
22     Node* y=NULL;
23
24     while(x!=NULL){
25         y=x;
26         if(z->key<x->key) x=x->left;

```

```

27     else x=x->right;
28 }
29
30 z->p=y;
31
32 if(y==NULL) T->root=z;
33 else if(z->key<y->key) y->left=z;
34 else y->right=z;
35 }
36
37 Node* treeSearch(Node* x,int k){
38     if(x==NULL || k==x->key) return x;
39     if(k<x->key) return treeSearch(x->left,k);
40     else return treeSearch(x->right,k);
41 }
42
43 Node* treeMin(Node* x){
44     while(x->left!=NULL) x=x->left;
45     return x;
46 }
47
48 void transplant(Tree* T,Node* u,Node* v){
49     if(u->p==NULL)
50         T->root=v;
51     else if(u==u->p->left)
52         u->p->left=v;
53     else
54         u->p->right=v;
55
56     if(v!=NULL)
57         v->p=u->p;
58 }
59
60 void treeDelete(Tree* T,Node* z){
61     Node* y;
62
63     if(z->left==NULL){
64         transplant(T,z,z->right);
65     }
66     else if(z->right==NULL){
67         transplant(T,z,z->left);
68     }
69     else{
70         y=treeMin(z->right);
71
72         if(y->p!=z){
73             transplant(T,y,y->right);
74             y->right=z->right;
75             y->right->p=y;
76         }
77
78         transplant(T,z,y);
79         y->left=z->left;
80         y->left->p=y;
81     }
82
83     free(z);
84 }
85
86 void inorder(Node* x){
87     if(x==NULL) return;
88     inorder(x->left);
89     printf("%d -> ",x->key);
90     inorder(x->right);
91 }
92
93 int main(){
94     int i;
95     int arr[]={50,30,70,20,40,60,80};
96     int deleteKeys[]={20,30,50};
97
98     Tree T;
99     T.root=NULL;
100
101     for(i=0;i<7;i++) insertNode(&T,arr[i]);
102
103     printf("Initial inorder traversal: ");
104     inorder(T.root);
105     printf("NULL\n");
106

```

```
107     for(i=0;i<3;i++){
108         Node* z=treeSearch(T.root,deleteKeys[i]);
109         if(z==NULL){
110             printf("\nKey %d not found in the tree.\n",deleteKeys[i]);
111         } else {
112             printf("\nDeleting %d:\n",deleteKeys[i]);
113             treeDelete(&T,z);
114             printf("Inorder traversal after deletion: ");
115             inorder(T.root);
116             printf("NULL\n");
117         }
118     }
119     return 0;
120 }
```

Rubric

Criteria	Marks
Correct program structure and function usage	2
Correct BST insertion and tree creation	2
Correct implementation of transplant() and deletion cases (leaf, one child, two children)	3
Proper maintenance of BST properties after deletion	2
Correct output	1
Total	10